2/10/26

# Lab 2 : Genetic Algorithm

Pseudocode for genetic algorithms

S1> Start

Step 2 : Initialize population of a random solution

Step 3 : Calculate fitness of each individual

Step 4 : Stop until condition of repeated satisfied

a> Find the best/fittest in population of solution

b> Make a pair of fittest in problem

c> Make a crossover of fittest individuals & form new offspring // crossovers

d> Perform mutation of random offspring by making some changes in gene // mutation

e> Calculate fitness of new offspring

f> Create a new population using selected individuals and offspring

Step 5 : Find fittest individual in group

Step 6 : Return best individual as an optimal solution of the problem

Step 7 : Stop

IEEE Papers: TSP

- Genetic Algorithm for Travelling salesman problem A review of representation & operators
- Proposed by Sahib Singh Tuneja, Sunil Choudhary

Summary :

- By genetic algorithms we have to calculate minimum possible distance & TSP for 20 cities & more cities TSP is hence considered an NP hard optimisation problem
- By this we have to generate routes, calculate distance. select good routes, crossover mutation new routes. repeat & test route can be found
- The paper tested the genetic Algorithm on problem involving different numbers of cities For 20 city problem, it obtained a best path of 552. matching best known path reported in the paper & that is achievement of it

Objective :

- The main objective of paper Travelling Salesman Problem is optimization using genetic algorithm is to find optimized route for the Travelling Salesman Problem (TSP) that visits all given cities & returns to starting city while minimizing the total travelling distance, using genetic algorithms
- The value of crossover probability (Pc) & mutation Probability (Pm) are key control parameters affecting the performance & congruence of GA

$$P_c = (K_1 (f_{max} - f) / (f_{max} - f_{avg}))$$

code :

```
import random
import math
def distance (city 1, city 2):
    return math.square((x2-x1) ** 2 + (y2-y1) ** 2)
def fitness (route):
def create population ():
def route selection (population):
def crossover (parent 1, parent 2):
def mutation ():
def genetic_algorithm ():
print (Best route)
```

output

Generation : 0 Best distance : 1092.23

Generation : 50 Best distance : 801.23

Final Result

Best Route

1> (0, 0)

2> (15, 23)

3> (10, 50)

Minimum distance : 801.23

Conl
8/10/26